ISTANBUL AREL UNIVERSITY
FACULTY OF ENGINEERING

LEEN364-DEEP LEARNING AND CLASSIFICATION TECHNIQUES

PROJECT REPORT

Project Title: Deep Learning-Based Arrhythmia Classification of Individual ECG Heartbeats

Name Surname: Ziya Ahmet Ertaul
Student Number: 230303056

1. INTRODUCTION

1.1 Objective

The objective of this project is to classify individual electrocardiogram (ECG) heartbeats into the five heartbeat classes recommended by the AAMI EC57 standard [1]: normal (N), supraventricular ectopic (S), ventricular ectopic (V), fusion (F) and unknown or paced (Q) beats. The models are trained and evaluated on the ECG Heartbeat Categorization dataset [2, 3], which contains segmented heartbeats from the MIT-BIH Arrhythmia Database [4]. Each heartbeat is given to the model as a raw signal of 187 samples, and the model predicts its class directly, without hand-crafted features.

About 83% of the beats in the dataset are normal, so a model can reach a high accuracy while missing most of the abnormal beats. The goal is therefore not only a high overall accuracy, but a reliable detection of the rare abnormal classes, especially S, V and F. For this reason the macro-averaged F1-score, which gives the same weight to every class, is used as the main evaluation metric.

1.2 Motivation

Arrhythmias are disturbances of the normal rhythm of the heart. Many of them appear only from time to time, so they are usually detected with long-term ECG recordings such as 24- to 48-hour Holter monitoring. At a typical heart rate of about 70 beats per minute, one day of recording contains around 100,000 heartbeats. Reviewing all of them manually is slow and tiring, and rare abnormal beats can easily be overlooked. An automatic heartbeat classifier can mark the suspicious beats, so that a cardiologist only needs to check a small part of the recording.

The problem is also interesting from a machine learning point of view. The classes are strongly imbalanced (the fusion class is less than 1% of the data), and some abnormal beats, in particular supraventricular ectopic beats, have almost the same shape as normal beats. This makes the task a good case study for comparing model architectures and methods for handling class imbalance.

1.3 Scope of the Project

The work in this project consists of the following parts:

- Two classical baselines (logistic regression and random forest) trained on the raw signal samples.
- Four neural networks: a fully connected network (MLP), a simple 1D convolutional neural network (CNN), a residual 1D CNN based on the architecture of Kachuee et al. [3], and a recurrent network with long short-term memory (LSTM) layers.
- A study of class-imbalance handling on the residual CNN: class weights, random oversampling, and oversampling combined with data augmentation.

- A careful evaluation protocol: all models and settings are selected on a validation set, the official test set is used only once at the end, and every final model is trained with three random seeds and reported as mean ± standard deviation.

The final model, a residual CNN trained with oversampling and data augmentation, reaches a macro F1-score of 0.925 ± 0.004 and a balanced accuracy of 93.5 ± 0.1% on the test set.

2. LITERATURE REVIEW

Automatic heartbeat classification has been studied for several decades, and most published work uses the MIT-BIH Arrhythmia Database [4] together with the five AAMI classes [1]. A detailed survey of the field is given by Luz et al. [5]. The existing methods can be divided into two groups: classical methods, which first extract hand-crafted features and then apply a classifier, and deep learning methods, which learn the features directly from the signal. The most relevant studies are summarized in Table 1.

2.1 Classical Methods

In the classical approach, each heartbeat is described by a small set of features chosen by experts. de Chazal et al. [6] combined morphology features of the QRS complex and the P and T waves with heartbeat interval (RR) features and used linear discriminant classifiers. The RR intervals are especially important for S beats, because a supraventricular ectopic beat often has a normal shape but comes earlier than expected. Other studies used the discrete wavelet transform to describe the shape of the beat at several time scales. Martis et al. [7] reduced wavelet coefficients with PCA, LDA and ICA and compared several classifiers, and Li and Zhou [8] used wavelet packet entropy and RR features with a random forest classifier. These methods are fast and easy to interpret, but their performance depends on the quality of the hand-crafted features and on an accurate detection of the ECG waves.

2.2 Deep Learning Methods

Deep neural networks remove the need for manual feature design. One-dimensional CNNs slide the same learned filters along the signal, so they can detect a local wave shape, such as a wide QRS complex, wherever it appears in the beat. Kiranyaz et al. [9] trained a compact patient-specific 1D CNN that can run in real time, and Acharya et al. [10] trained a nine-layer CNN on MIT-BIH heartbeats and generated synthetic data to balance the classes. Kachuee et al. [3] proposed a deep residual CNN, built from residual blocks with skip connections [11], and reported an average accuracy of 93.4% for the five AAMI classes. They also showed that the learned representation can be transferred to myocardial infarction classification. Their preprocessed heartbeats form the dataset used in this project. On a much larger scale, Hannun et al. [12] trained a 34-layer CNN on 91,232 single-lead ECG records from 53,549 patients and reached a performance similar to that of cardiologists for 12 rhythm classes.

Since an ECG is a time series, recurrent neural networks are a natural alternative. Long short-term memory (LSTM) networks [13] use gates to keep information over many time steps. Yildirim [14] combined a wavelet-based sequence representation with a deep bidirectional LSTM and reported a recognition rate of 99.39% for the beat types considered in that study. A drawback of LSTMs is that they process the signal one time step after another, which makes training much slower than for CNNs.

Table 1. Summary of related work on heartbeat classification.

Study	Features and model	Remarks
de Chazal et al. [6]	Morphology and RR interval features, linear discriminants	Inter-patient evaluation
Martis et al. [7]	Wavelet coefficients reduced with PCA, LDA or ICA; several classifiers	Compares feature reduction methods
Li and Zhou [8]	Wavelet packet entropy and RR features, random forest	Inter-patient evaluation
Kiranyaz et al. [9]	Compact 1D CNN	Patient-specific, real-time
Acharya et al. [10]	Nine-layer 1D CNN	Synthetic data to balance the classes
Kachuee et al. [3]	Residual 1D CNN	93.4% average accuracy on a class-balanced test set
Yildirim [14]	Wavelet sequence and bidirectional LSTM	Recurrent model
Hannun et al. [12]	34-layer CNN on 53,549 patients	Cardiologist-level, 12 rhythm classes

2.3 Evaluation Paradigms

An important difference between studies is the way the data are divided into training and test sets. In the *intra-patient* (beat-based) paradigm, beats of the same patient can appear in both sets. In the *inter-patient* paradigm, used for example by de Chazal et al. [6], the training and test beats come from different patients. Luz et al. [5] point out that intra-patient results are usually much higher and are too optimistic for clinical use, because the model has already seen beats of the test patients during training. The test sets also differ: the 93.4% of Kachuee et al. [3], for example, was measured on a test set with about the same number of beats from each class. For these reasons, results from different studies can only be compared with care.

2.4 Position of This Project

This project builds on the residual CNN of Kachuee et al. [3] and uses the same preprocessed data. Compared to that work, it (i) compares the CNN with classical baselines, an MLP and an LSTM under the same training protocol, (ii) studies the effect of class weights, oversampling and data augmentation separately, and (iii) evaluates the models on the original, imbalanced test set with the macro F1-score and balanced accuracy, using

three random seeds. The dataset uses a beat-based split, so the results of this project are intra-patient results.

3. DATASET DESCRIPTION

3.1 Dataset Name and Source

The project uses the MIT-BIH part of the **ECG Heartbeat Categorization Dataset**, which is publicly available on Kaggle [2] and was prepared by Kachuee et al. [3]. It is derived from the MIT-BIH Arrhythmia Database [4], which contains 48 half-hour, two-channel ECG recordings from 47 subjects, originally sampled at 360 Hz. Every beat in the database was annotated by cardiologists. The dataset is given as two CSV files, `mitbih_train.csv` and `mitbih_test.csv`, with 87,554 and 21,892 heartbeats (109,446 in total).

Kachuee et al. [3] extracted the heartbeats from ECG lead II, resampled to 125 Hz, with the following steps: the signal is split into 10-second windows and normalized to the range [0, 1]; R-peaks are found as local maxima above 0.9; the median R-R interval of the window is taken as the nominal heartbeat period T; and for each R-peak a segment of length 1.2T is cut out and padded with zeros to a fixed length. No filtering was applied.

3.2 Features and Labels

Each row of the CSV files is one heartbeat. The first 187 columns are the signal samples (about 1.5 s at 125 Hz), with values between 0 and 1, and the last column is the class label (0 to 4). Every beat starts at its R-peak, and beats shorter than 187 samples are padded with zeros at the end. The five classes follow the AAMI EC57 recommendation [1] and group the original MIT-BIH beat types as shown in Table 2.

Table 2. The five AAMI heartbeat classes and the number of beats in the official training and test files.

Label	Class	MIT-BIH beat types	Train	Test
0	N – Normal	Normal, left and right bundle branch block, atrial escape, nodal escape	72,471 (82.8%)	18,118 (82.8%)
1	S – Supraventricular ectopic	Atrial premature, aberrant atrial premature, nodal premature, supraventricular premature	2,223 (2.5%)	556 (2.5%)
2	V – Ventricular ectopic	Premature ventricular contraction, ventricular escape	5,788 (6.6%)	1,448 (6.6%)
3	F – Fusion	Fusion of ventricular and normal beat	641 (0.7%)	162 (0.7%)
4	Q – Unknown	Paced, fusion of paced and normal, unclassifiable	6,431 (7.3%)	1,608 (7.3%)

3.3 Exploratory Data Analysis

Figure 1 shows the class distribution. The dataset is strongly imbalanced: 82.8% of the beats are normal, while the fusion class has only 641 training beats (0.7%). A trivial model that always predicts N would already reach about 83% accuracy, which is why accuracy alone is not a good measure of performance for this problem. The training and test files have almost exactly the same class proportions.

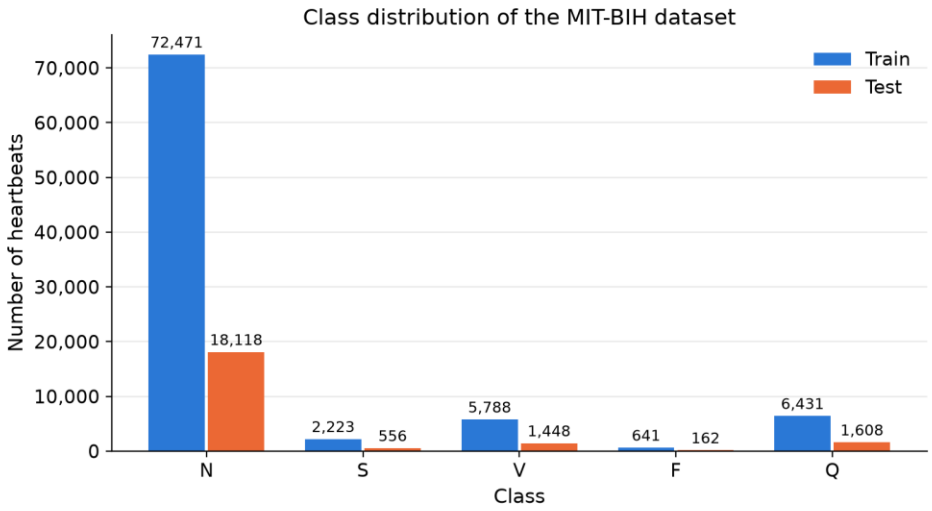


Figure 1. Number of heartbeats per class in the official training and test files.

Figure 2 shows randomly selected training beats from each class, and Figure 3 shows the mean beat of each class with a band of one standard deviation. Several properties of the data can be seen in these figures:

- Every beat starts with a high R-peak, and the R-peak of the next beat often appears near the end of the signal, followed by the zero padding.
- V beats usually have a wide, large deflection after the R-peak, and Q beats (mostly paced beats) have a very regular shape with a small standard deviation.
- The mean N and S beats are very similar. S beats differ from N beats mainly in their timing, not in their shape, which makes them hard to recognize from a single beat.
- The beats have different effective lengths. The mean length before the zero padding is 0.89 s for N, 0.91 s for S, 0.95 s for V and Q, and only 0.63 s for F, because the segment length depends on the heart rate of the recording.

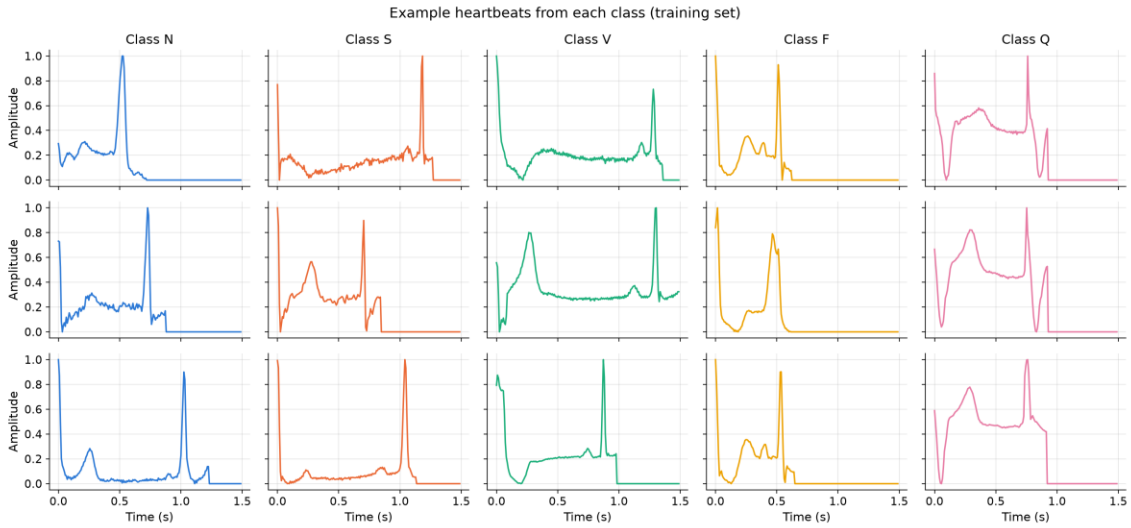


Figure 2. Three randomly selected training heartbeats from each class.

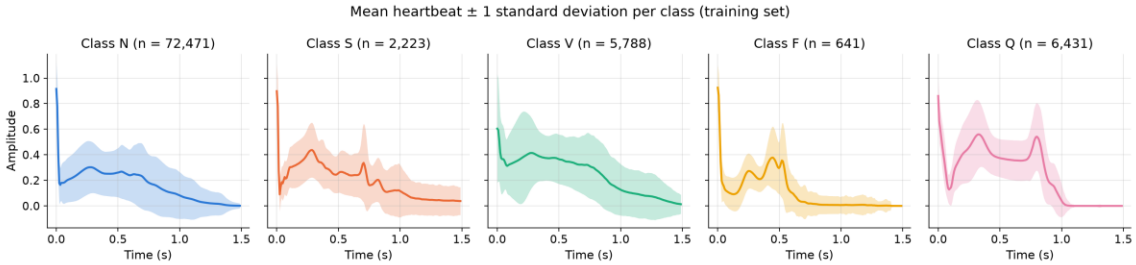


Figure 3. Mean heartbeat of each class (line) with ±1 standard deviation (band), computed on the training file. The zero padding is included in the mean.

3.4 Data Quality Checks

Before modelling, both files were checked for problems. There are no missing or infinite values, all values lie in the range [0, 1], and there are no empty, constant or duplicated beats. There are also no beats that appear in both the training and the test file, so there is no direct leakage between them.

3.5 Preprocessing and Data Split

Since the beats are already segmented, aligned to the R-peak and normalized, no further filtering or normalization was applied to the signals. The label column was converted to integers. For the logistic regression model, the samples were additionally standardized with a scaler fitted on the training data only.

The official test file was kept untouched until the final evaluation. From the official training file, 20% of the beats were held out as a stratified validation set (random seed 42), which keeps the class proportions the same in both parts (Table 3). All model choices, hyperparameters and the early stopping of the neural networks are based on this validation

set. Oversampling and data augmentation, described in Section 4, were applied to the training part only.

Table 3. Number of heartbeats per class in the training, validation and test sets.

Class	Training	Validation	Test
N	57,977	14,494	18,118
S	1,778	445	556
V	4,630	1,158	1,448
F	513	128	162
Q	5,145	1,286	1,608
Total	**70,043**	**17,511**	**21,892**

A known limitation of this dataset is that the split is based on beats and not on patients. Beats of the same patient can therefore appear in both the training and the test set, so the results describe an intra-patient setting (Section 2.3) and are likely to be higher than on unseen patients.

4. METHODOLOGY

The project follows a step-by-step approach. First, two classical baselines are trained to show what can be reached without deep learning. Then four neural networks of increasing complexity are compared under the same training settings. The best architecture is used to study different ways of handling the class imbalance. Finally, the selected models are retrained with three random seeds and evaluated once on the test set. All choices in the first three steps are made on the validation set.

4.1 Model Selection

Classical baselines. Logistic regression and random forest classifiers are trained directly on the 187 signal samples, without any feature extraction. Logistic regression is a linear model, and its inputs are standardized (zero mean and unit variance, fitted on the training data). The random forest uses 200 decision trees, each trained on a bootstrap sample of the data; it does not need feature scaling and can model non-linear relations. Both models were trained with and without balanced class weights, and the version with the higher validation macro F1-score was kept: logistic regression without class weights and random forest with balanced class weights.

Neural networks. Four architectures were implemented (Table 4):

- **MLP:** a fully connected network. Every input sample is connected to every neuron, so the network has no notion of the order of the samples or of local wave shapes. It is the simplest neural baseline.

- **Simple 1D CNN:** three convolution blocks with 32, 64 and 128 filters of size 5, each followed by max pooling. A convolution filter slides along the signal, so the same local pattern, for example a sharp QRS complex, is recognized wherever it appears in the beat.
- **Residual 1D CNN:** the main model, based on the architecture of Kachuee et al. [3]. After a first convolution layer, five residual blocks are stacked. Each block contains two convolution layers (32 filters of size 5), a skip connection that adds the block input to the output, a ReLU activation and max pooling with size 5 and stride 2. The skip connection [11] lets each block learn only a correction of its input and helps the gradients reach the first layers. The pooling layers reduce the signal length from 187 to 92, 44, 20, 8 and finally 2 time steps.
- **LSTM:** two stacked LSTM layers [13] with 64 units read the beat one sample at a time. The first layer passes its full output sequence to the second, and the final state of the second layer is used for classification.

Table 4. Architectures of the neural networks. All models end with a dense layer of 5 units and softmax activation.

Model	Layers	Parameters
MLP	Dense(256, ReLU) → Dropout(0.3) → Dense(128, ReLU) → Dropout(0.3) → Dense(5)	81,669
Simple 1D CNN	3 × [Conv1D(32 / 64 / 128 filters, size 5, ReLU) → MaxPool(2)] → Flatten → Dense(64, ReLU) → Dropout(0.3) → Dense(5)	240,389
Residual 1D CNN	Conv1D(32, size 5) → 5 × residual block → Flatten → Dense(32, ReLU) → Dense(5)	53,957
LSTM	LSTM(64, full sequence) → LSTM(64) → Dense(32, ReLU) → Dense(5)	52,165

4.2 Training Procedure and Hyperparameters

All neural networks are trained with the same settings, summarized in Table 5. The loss function is the categorical cross-entropy and the optimizer is Adam [15]. Two callbacks control the training. **Early stopping** computes the macro F1-score on the validation set after every epoch and stops the training when it has not improved for 8 epochs; the weights of the best epoch are then restored. The macro F1-score was used instead of the validation loss because it is the metric that the project aims to maximize. **ReduceLROnPlateau** halves the learning rate when the validation loss has not improved for 4 epochs, which allows smaller and more careful weight updates near the end of the training.

Table 5. Training settings used for all neural networks.

Setting	Value
Optimizer	Adam, initial learning rate 0.001
Loss function	Sparse categorical cross-entropy

Setting	Value
Batch size	128
Maximum number of epochs	50
Early stopping	Validation macro F1-score, patience 8, best weights restored
Learning rate schedule	Halved when validation loss does not improve for 4 epochs
Dropout	0.3 (MLP and simple CNN)
Random seeds	42 (model development); 42, 123 and 2026 (final evaluation)

4.3 Handling the Class Imbalance

Without any correction, a model trained on this data sees about 113 normal beats for every fusion beat, so it can reach a low loss while largely ignoring the rare classes. Three methods were tested on the residual CNN (Table 6). In every experiment, only the training part was changed; the validation set was always the original one.

- **Class weights:** the loss of each training beat is multiplied by the weight of its class, so a mistake on a rare class costs more. The weights are $w_c = n / (5 \cdot n_c)$, where n is the number of training beats and n_c the number of beats of class c. This gives weights of 0.24 for N, 7.88 for S, 3.03 for V, 27.31 for F and 2.72 for Q.
- **Random oversampling:** the beats of every minority class are copied (sampled with replacement) until each class has as many training beats as N (57,977). The training set grows from 70,043 to 289,885 beats, so the network sees all classes equally often.
- **Data augmentation:** each training batch is changed randomly on the fly, so every copy of a duplicated beat looks slightly different in every epoch. Three transformations are each applied with a probability of 0.5: a *time shift* of up to ±5 samples, an *amplitude scaling* by a random factor between 0.9 and 1.1, and *Gaussian noise* with a standard deviation of 0.02. The shift is applied inside the beat, and the noise is added only to the beat and not to the zero padding, so the padding remains exactly zero as in the real data (Figure 4).



Figure 4. Effect of the three augmentation methods on one training beat of class S (grey: original, blue: augmented).

Table 6. Class-imbalance experiments on the residual CNN.

Experiment	Class weights	Oversampling	Augmentation
A (reference)	–	–	–
B	Balanced	–	–
C	–	All classes to 57,977 beats	–
D	–	All classes to 57,977 beats	Shift, scaling and noise
D2	–	All classes to 57,977 beats	Scaling and noise (no shift)

Following the plan, experiment D combines augmentation with the better of B and C. Experiment D2 was added after D to test whether the time shift is helpful: since every beat starts at its R-peak, a shift either cuts the peak off or adds a flat segment at the start, which never occurs in real beats.

4.4 Evaluation Metrics and Model Selection

For each class c, the precision is TP / (TP + FP), the recall (sensitivity) is TP / (TP + FN), and the F1-score is the harmonic mean of the two, where TP, FP and FN are the true positives, false positives and false negatives of that class. The following overall metrics are reported:

- **Macro F1-score** (main metric): the unweighted mean of the five per-class F1-scores. Each class counts equally, so the score drops clearly if a rare class is classified badly.
- **Balanced accuracy:** the mean of the five per-class recalls.
- **Accuracy:** the fraction of correctly classified beats. It is reported for completeness, but it is dominated by the N class.
- **Per-class recall of S, V and F**, and row-normalized **confusion matrices**, which show how the beats of each true class are distributed over the predicted classes.

The configuration for the final model was chosen with a rule that was fixed before the last experiment was run: the configuration with the highest validation macro F1-score is selected, but if the two best differ by less than 0.005, the difference is treated as noise from a single training run and the configuration with less overfitting (lower validation loss in the last epochs) is selected.

For the final evaluation, every selected model is trained with the seeds 42, 123 and 2026, which change the initial weights, the order of the training batches and the random oversampling and augmentation. The validation split stays the same. A training run is counted as failed if it never leaves the trivial solution of predicting N for every beat; this is decided on the validation set (macro F1-score below 0.3, while predicting only N gives 0.18). The test set is used once, after all models have been trained, and the results are reported as mean ± standard deviation over the seeds.

4.5 Frameworks and Tools

The project is implemented in Python 3.13. The neural networks are built and trained with TensorFlow 2.21 [16] and its Keras 3.15 API [17]. The classical models, the metrics and the validation split use scikit-learn 1.9.1 [18], and random oversampling uses imbalanced-learn 0.14.2 [19]. NumPy and pandas are used for data handling, and Matplotlib and seaborn for the figures. All models were trained on a CPU.

Assistance was received from Claude AI [20] during the project.

5. CODE IMPLEMENTATION

The complete code, the notebooks with all their outputs, the figures and the saved results of every experiment are available on GitHub: **https://github.com/ahmetertaul/ecg-project**

5.1 Code Structure

The reusable code is organized as a small Python package (`src`), and the experiments are run in five Jupyter notebooks that import it (Table 7). The settings and results of every experiment are saved as JSON files in `results/metrics`, and all figures are saved in `results/figures`, so every number in this report can be traced back to a file.

Table 7. Files of the project and their purpose.

File	Purpose
`src/data.py`	Loading the CSV files and the stratified train/validation split
`src/models.py`	Definitions of the MLP, simple CNN, residual CNN and LSTM
`src/train.py`	Training loop with early stopping on the validation macro F1-score
`src/augment.py`	Time shift, amplitude scaling and noise augmentation of training batches
`src/evaluate.py`	Metrics, saving of results and plotting functions
`src/run_final.py`	Training of the final models with three seeds
`notebooks/01_eda.ipynb`	Exploratory data analysis and data quality checks
`notebooks/02_baselines.ipynb`	Logistic regression and random forest
`notebooks/03_deep_models.ipynb`	MLP, simple CNN, residual CNN and LSTM
`notebooks/04_imbalance.ipynb`	Class weights, oversampling and augmentation experiments
`notebooks/05_final_evaluation.ipynb`	Single evaluation of the final models on the test set

5.2 Data Loading and Preprocessing

Each CSV file is read with pandas. The first 187 columns become the signal matrix and the last column the integer label. The validation set is split off with scikit-learn, stratified by class:

```
def load_mitbih(split="train", data_dir=DATA_DIR):
    df = pd.read_csv(Path(data_dir) / f"mitbih_{split}.csv", header=None)
    X = df.iloc[:, :-1].to_numpy(dtype=np.float32)
    y = df.iloc[:, -1].to_numpy().astype(np.int64)
    return X, y


def split_train_val(X, y, val_size=0.2, seed=SEED):
    return train_test_split(X, y, test_size=val_size, stratify=y,
                            random_state=seed)
```

5.3 Model Definition and Compilation

The models are built with the Keras functional API. The residual block of the main model adds the block input to the output of two convolution layers before the activation and the pooling:

```
def residual_block(x, filters=32, kernel_size=5):
    shortcut = x
    y = layers.Conv1D(filters, kernel_size, padding="same", activation="relu")(x)
    y = layers.Conv1D(filters, kernel_size, padding="same")(y)
    y = layers.Add()([y, shortcut])
    y = layers.Activation("relu")(y)
    return layers.MaxPooling1D(pool_size=5, strides=2)(y)
```

All networks are compiled in the same way:

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

5.4 Model Training and Evaluation

A custom Keras callback computes the macro F1-score on the validation set after each epoch, so that early stopping can monitor it. For experiment D, the training data are first oversampled with imbalanced-learn and then passed through a data generator (`AugmentedBatches`) that augments every batch on the fly:

```
callbacks = [
    MacroF1Callback(X_val, y_val),
    keras.callbacks.EarlyStopping(monitor="val_macro_f1", mode="max",
                                  patience=patience, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
                                      patience=4),
]
```

After training, the predictions on the validation (or, at the end, the test) set are passed to `compute_metrics`, which returns the accuracy, balanced accuracy, macro F1-score and the per-class precision, recall and F1-score. These metrics, the hyperparameters and the training history are saved together with `save_results`, and the plots in Section 6 are produced with `plot_history` and `plot_confusion_matrix`. The script `run_final.py` repeats the training for all final models and seeds and can be restarted, since it skips runs whose results already exist.

6. RESULTS AND ANALYSIS

The results are presented in the order of the experiments. Sections 6.1 to 6.3 report results on the validation set, which were used to make all modelling decisions (seed 42). Sections 6.4 to 6.7 report the final results on the test set.

6.1 Comparison of Models on the Validation Set

Table 8 compares all models trained without imbalance handling, except for the class-weighted baselines. Logistic regression performs poorly (macro F1-score 0.657): a linear model on raw samples cannot separate the classes, and it finds only 34% of the V beats and 28% of the F beats. With balanced class weights its recall improves, but its precision collapses (only 15% of its S predictions and 8% of its F predictions are correct), so the macro F1-score drops to 0.482. The random forest is much stronger and benefits from class weights (0.876 to 0.900).

Table 8. Validation results of the baselines and the neural networks (seed 42). The best value in each column is shown in bold.

Model	Accuracy	Balanced acc.	Macro F1	Recall S	Recall V	Recall F
Logistic regression	0.914	0.588	0.657	0.449	0.337	0.281
Logistic regression (balanced)	0.675	0.772	0.482	0.692	0.739	**0.852**
Random forest	0.976	0.815	0.876	0.620	0.892	0.609
Random forest (balanced)	0.980	0.869	0.900	0.746	0.936	0.695
MLP	0.981	0.870	0.904	0.708	0.934	0.727
Simple 1D CNN	0.988	0.909	0.929	0.820	0.964	0.773
Residual 1D CNN	**0.989**	**0.920**	**0.933**	**0.872**	**0.979**	0.758
LSTM	0.980	0.861	0.896	0.692	0.945	0.688

Among the neural networks, both CNNs are clearly better than the MLP and the LSTM, and the residual CNN is the best model in almost every column. The improvement is largest for the S class (recall 0.708 for the MLP and 0.872 for the residual CNN). The LSTM reaches only the level of the MLP and was by far the slowest model: one epoch took about 70 s, compared to about 7 s for the residual CNN, and training lasted about 56 minutes. For

all models, F is the hardest class: even the residual CNN finds only about three quarters of the fusion beats.

6.2 Training Curves

Figure 5 shows the training curves of the residual CNN. The training accuracy reaches almost 100%, while the validation loss reaches its minimum around epoch 15 and then increases again. This is a typical sign of overfitting: the network starts to memorize the training beats. Early stopping on the validation macro F1-score selected epoch 25 and stopped the training at epoch 33.

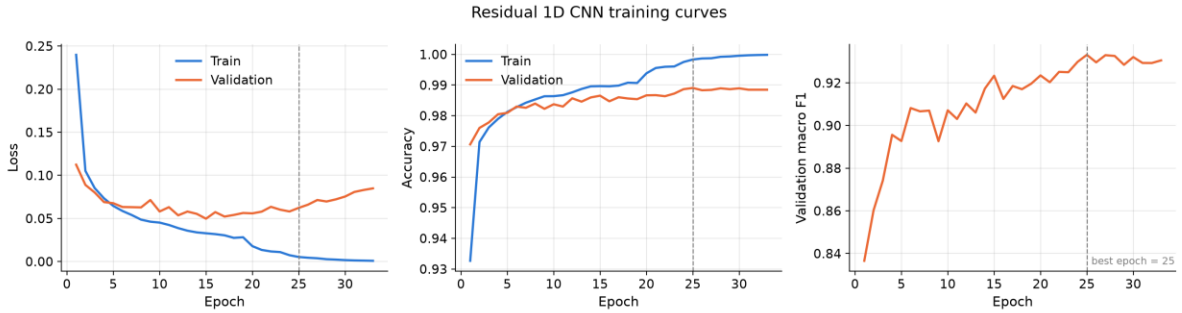


Figure 5. Training curves of the residual CNN without imbalance handling (experiment A). The dashed line marks the epoch with the best validation macro F1-score.

The LSTM (Figure 6) behaves very differently. During the first four epochs it predicts N for every beat (validation macro F1-score 0.18), and it only then starts to learn the other classes. Its curves show sudden jumps around epochs 14 and 19, which become smoother after the learning rate is halved at epoch 20. The training and validation curves stay close together, so the LSTM does not overfit. However, its best epoch was epoch 48 of 50, so it would probably have improved slightly with a longer training.

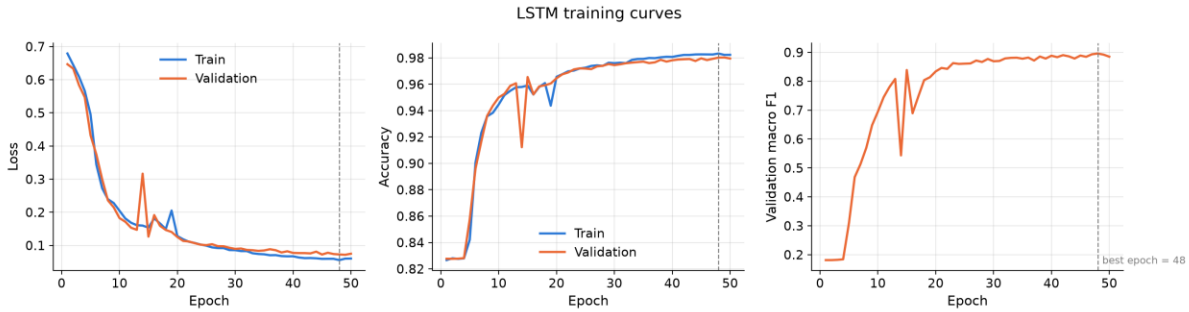


Figure 6. Training curves of the LSTM. The network predicts only class N during the first four epochs.

6.3 Class-Imbalance Experiments

Table 9 shows the results of the class-imbalance experiments on the residual CNN.

Table 9. Validation results of the class-imbalance experiments on the residual CNN (seed 42). The last column is the mean validation loss over the last five epochs.

Experiment	Macro F1	Balanced acc.	Recall S	Recall F	Precision F	Final val. loss
A: none	0.933	0.920	0.872	0.758	**0.890**	**0.079**
B: class weights	0.917	0.933	0.872	**0.844**	0.730	0.096
C: oversampling	**0.936**	0.931	0.874	0.828	0.828	0.151
D: C + augmentation	0.933	**0.937**	**0.885**	**0.844**	0.812	0.082
D2: D without shift	0.932	0.929	0.874	0.812	0.806	0.119

The results show four main points:

- **Class weights (B)** increased the recall of F from 0.758 to 0.844, but the network then predicted the rare classes too often: the precision of F fell from 0.890 to 0.730, and the macro F1-score decreased to 0.917. With a weight of 27 for F, a single fusion beat in a batch strongly changes the gradient, which makes the training less smooth.
- **Oversampling (C)** gave the highest macro F1-score (0.936) and a balanced precision and recall for F (both 0.828). However, the validation loss increased from 0.081 at epoch 11 to 0.159 at the end of the training, while the training accuracy reached 100%. Each fusion beat appears about 113 times per epoch, so the network memorizes these copies.
- **Data augmentation (D)** stopped this overfitting: the validation loss stayed around 0.08 (Figure 7). D also gave the highest balanced accuracy (0.937) and the highest recall for S and F.
- **Removing the time shift (D2)** brought the overfitting back (final validation loss 0.119). Although the shift creates beats that do not occur in real data, it was the augmentation that prevented the network from memorizing the exact position of the waves.

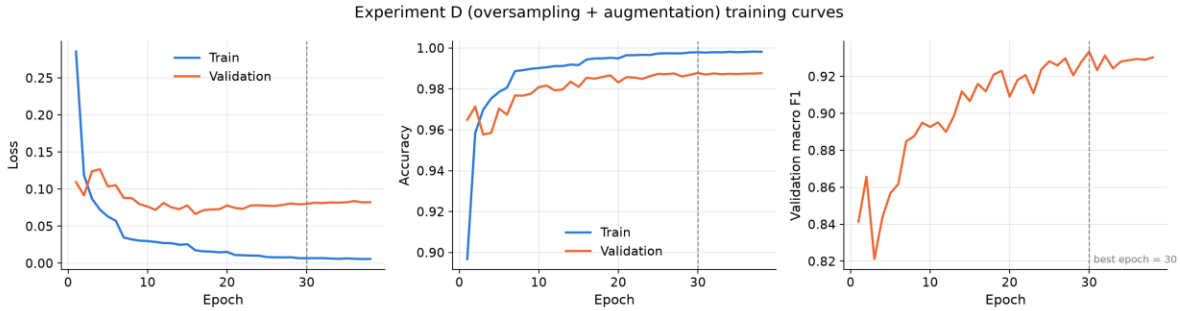


Figure 7. Training curves of the residual CNN with oversampling and data augmentation (experiment D).

The two best configurations by validation macro F1-score were C (0.936) and D (0.933). Their difference (0.003) is smaller than the threshold of 0.005, so by the selection rule (Section 4.4) configuration **D** was selected as the final model because it overfits much less.

6.4 Final Results on the Test Set

Table 10 shows the final results on the test set as mean ± standard deviation over the three seeds, and Figure 8 compares the macro F1-scores. The ranking of the models is the same as on the validation set.

Table 10. Test-set results (mean ± standard deviation over three seeds). The best value in each column is shown in bold.

Model	Accuracy	Balanced acc.	Macro F1	Recall S	Recall V	Recall F
Logistic regression	0.915 ± 0.000	0.597 ± 0.000	0.664 ± 0.000	0.421	0.334	0.364
Random forest (bal.)	0.977 ± 0.000	0.865 ± 0.002	0.890 ± 0.001	0.714	0.926	0.730
MLP	0.980 ± 0.000	0.857 ± 0.006	0.893 ± 0.004	0.671	0.935	0.708
Simple 1D CNN	0.985 ± 0.000	0.889 ± 0.001	0.915 ± 0.004	0.779	0.956	0.730
Residual CNN (A)	0.985 ± 0.004	0.904 ± 0.016	0.918 ± 0.015	0.800	**0.964**	0.772
Residual CNN (D)	**0.986 ± 0.001**	**0.935 ± 0.001**	**0.925 ± 0.004**	**0.864**	0.959	**0.868**
LSTM[1]	0.976 ± 0.002	0.836 ± 0.016	0.875 ± 0.007	0.648	0.912	0.660

[1] Mean of two runs. With seed 123, the LSTM never left the trivial solution and predicted N for every beat (validation macro F1-score 0.184, test macro F1-score 0.185). This run was excluded based on the validation criterion in Section 4.4, but it shows that the LSTM training is unstable.

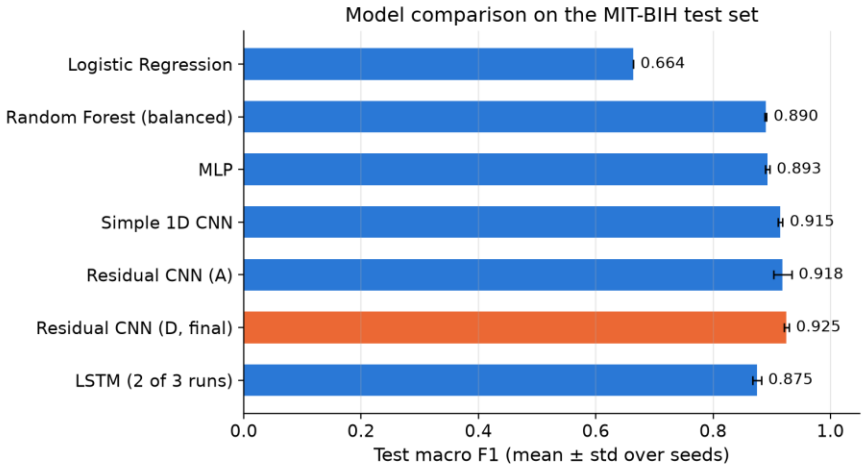


Figure 8. Test macro F1-score of all models (mean ± standard deviation over three seeds). The final model is shown in orange.

The final residual CNN (D) is the best model with a macro F1-score of 0.925 ± 0.004 and a balanced accuracy of 0.935 ± 0.001. Compared to the same network without imbalance handling (A), the gain in macro F1-score is small (+0.007), but the recall of the rare classes increases clearly: from 0.800 to 0.864 for S and from 0.772 to 0.868 for F. The final model is also much more stable: the standard deviation of the macro F1-score over the seeds is 0.004, compared to 0.015 for A. The random forest and all neural networks score only 0.007 to 0.016 lower on the test set than on the validation set (logistic regression even scores slightly higher), so the model choices were not overfitted to the validation set.

6.5 Confusion Matrix

Figure 9 shows the row-normalized confusion matrix of the final model, with the predictions of the three seeds pooled, and Table 11 gives the per-class metrics. N, V and Q beats are recognized almost perfectly. The two main error types are S beats predicted as N (12%) and F beats predicted as V (7%) or N (5%). The first error agrees with the observation in Section 3.3 that S beats have almost the same shape as N beats. The second error is expected, because a fusion beat is by definition a mixture of a ventricular and a normal beat.

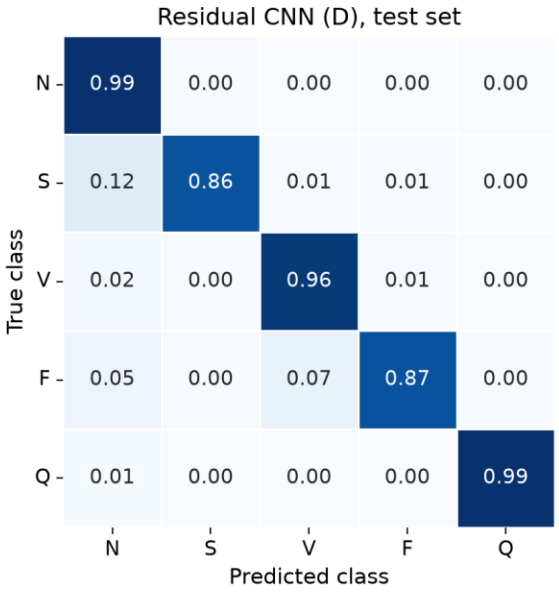


Figure 9. Row-normalized confusion matrix of the final model on the test set (three seeds pooled).

Table 11. Per-class test results of the final model (mean over three seeds).

Class	Precision	Recall	F1-score	Test beats
N	0.993	0.993	0.993	18,118
S	0.845	0.864	0.854	556
V	0.965	0.959	0.962	1,448
F	0.784	0.868	0.824	162

Class	Precision	Recall	F1-score	Test beats
Q	0.995	0.990	0.993	1,608

6.6 Example Predictions

Figure 10 shows one correctly and one incorrectly classified test beat of each class, chosen at random. The misclassified S beat does not start with a clear R-peak, unlike most beats in the dataset, and the misclassified F beat has a large, sharp peak similar to a V beat. The misclassified Q beat is an almost flat, unusual signal. Several of the errors are therefore beats with an atypical shape or segmentation.

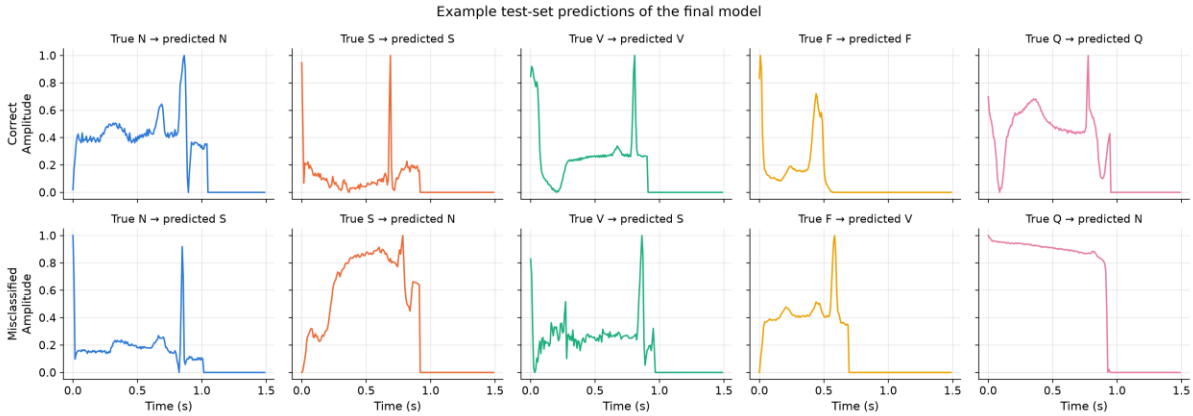


Figure 10. Test beats classified by the final model (seed 42). Top row: correct predictions; bottom row: incorrect predictions.

6.7 Comparison with the Literature

Kachuee et al. [3] reported an average accuracy of 93.4% for the same five classes. Their test set contained about 819 beats of each class, while the official test set used here is strongly imbalanced. On a balanced test set, the accuracy is equal to the mean of the per-class recalls, which is the balanced accuracy. The balanced accuracy is therefore the most comparable metric (Table 12). Our final model reaches 93.5 ± 0.1%, which is at the same level as the result of Kachuee et al. The plain accuracy of 98.6% is higher only because most test beats are normal, and it should not be compared with their number.

Table 12. Comparison with Kachuee et al. [3].

Study	Test set	Metric	Value
Kachuee et al. [3]	4,079 beats, about 819 per class	Average accuracy	93.4%
This work, residual CNN (D)	21,892 beats, official split	Accuracy	98.6 ± 0.1%
This work, residual CNN (D)	21,892 beats, official split	Balanced accuracy	93.5 ± 0.1%

7. DISCUSSION AND CONCLUSION

7.1 Performance Evaluation

At the start of the project, three outcomes were expected: (i) the neural networks would outperform the classical baselines, (ii) the CNNs would perform better than the MLP and the LSTM, because the classes differ mainly in the local shape of the waves, and (iii) handling the class imbalance would improve the macro F1-score. The first two expectations were confirmed, and the third only partly.

Comparison with the baselines. The final model improves the macro F1-score from 0.890 (random forest, the best baseline) to 0.925, and the largest gains are in the rare classes: the recall of S increases from 0.714 to 0.864 and the recall of F from 0.730 to 0.868. Logistic regression (0.664) shows that a linear model on the raw samples is not sufficient for this task.

Why the CNNs perform best. A convolution filter detects the same local pattern at any position in the beat and shares its weights over the whole signal. This matches the problem well, since the position of the waves varies from beat to beat while their shape carries the class information. The residual CNN is also efficient: with 53,957 parameters it is more than four times smaller than the simple CNN (240,389 parameters), but more accurate. The MLP, in contrast, must learn every pattern separately for each position. The LSTM can in principle model the whole sequence, but it has to carry the information through 187 time steps. It was about ten times slower per epoch than the residual CNN, reached only the level of the MLP, and one of its three training runs failed completely.

Effect of the class-imbalance handling. On the test set, oversampling with augmentation improved the macro F1-score of the residual CNN only from 0.918 to 0.925. Its main effects were elsewhere: the recall of S and F increased by 6 and 10 percentage points, the balanced accuracy increased from 0.904 to 0.935, and the results became much more stable across seeds. The macro F1-score changes little because the higher recall of the rare classes is partly paid for with a lower precision. In a screening application, a missed abnormal beat is usually more costly than a false alarm that a cardiologist can quickly dismiss, so this trade-off is favourable. Class weights, on the other hand, moved the balance too far towards the rare classes and reduced the macro F1-score.

Comparison with the literature. With a balanced accuracy of $93.5 \pm 0.1\%$, the final model reaches the same level as the 93.4% average accuracy reported by Kachuee et al. [3] for a similar residual CNN, although the test sets are not identical (Section 6.7).

7.2 Challenges

- **Class imbalance.** The F class has only 513 training beats. Even the final model reaches an F1-score of only 0.824 for F, the lowest of all classes. Simple

oversampling led to memorization of the copied beats, which had to be solved with data augmentation.

- **Similarity of N and S beats.** About 12% of the S beats are still classified as N. An S beat is mainly defined by coming too early, which is a property of the rhythm and not of the beat shape. The model only sees a single beat and has no information about the previous R-R interval, which classical methods use as a key feature [6].
- **Overfitting.** The residual CNN reached almost 100% training accuracy within a few epochs. Early stopping on the validation macro F1-score and data augmentation were needed to control it. The experiment without the time shift showed that the most useful augmentation was not the most realistic one.
- **Training cost and stability of the LSTM.** All models were trained on a CPU, because TensorFlow does not support GPUs on native Windows. Training the final models with the two additional seeds took about two hours, 70% of it for the LSTM, and one LSTM run never learned to predict any class other than N.
- **Beat-based split.** The dataset does not contain patient identifiers, so beats of the same patient can be in both the training and the test set. The reported results are therefore intra-patient results and are likely to be optimistic for patients that the model has never seen [5].

7.3 Future Work

- Evaluate the model with an inter-patient split, using the original MIT-BIH records and the division into training and test patients proposed by de Chazal et al. [6].
- Give the network information about the rhythm, for example the previous and next R-R intervals or the neighbouring beats, to reduce the confusion between N and S.
- Train the LSTM with techniques that make recurrent networks more stable, or combine convolution and LSTM layers in a hybrid model.
- Visualize which parts of the beat the CNN uses for its decision, for example with saliency maps.

7.4 Conclusion

In this project, individual ECG heartbeats from the MIT-BIH Arrhythmia Database were classified into the five AAMI classes with classical machine learning models and four neural network architectures. A residual 1D CNN, trained with random oversampling and data augmentation, gave the best results on the official test set: a macro F1-score of 0.925 $\pm$ 0.004, a balanced accuracy of 93.5 $\pm$ 0.1% and recalls of 0.864 and 0.868 for the rare S and F classes. CNNs clearly outperformed the classical baselines, the MLP and the LSTM. The study of the class imbalance showed that the main benefit of oversampling and augmentation is a better detection of the rare classes and more stable training, rather than a higher macro F1-score. The remaining errors are mostly S beats that look like normal

beats, which suggests that information about the heart rhythm is needed to improve the model further.

8. REFERENCES

[1] ANSI/AAMI EC57:2012, *Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms*. Arlington, VA, USA: Association for the Advancement of Medical Instrumentation, 2012.

[2] S. Fazeli, "ECG Heartbeat Categorization Dataset," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/datasets/shayanfazeli/heartbeat

[3] M. Kachuee, S. Fazeli, and M. Sarrafzadeh, "ECG heartbeat classification: A deep transferable representation," in *Proc. IEEE Int. Conf. Healthcare Informatics (ICHI)*, 2018, pp. 443–444.

[4] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH Arrhythmia Database," *IEEE Eng. Med. Biol. Mag.*, vol. 20, no. 3, pp. 45–50, 2001.

[5] E. J. da S. Luz, W. R. Schwartz, G. Cámara-Chávez, and D. Menotti, "ECG-based heartbeat classification for arrhythmia detection: A survey," *Comput. Methods Programs Biomed.*, vol. 127, pp. 144–164, 2016.

[6] P. de Chazal, M. O'Dwyer, and R. B. Reilly, "Automatic classification of heartbeats using ECG morphology and heartbeat interval features," *IEEE Trans. Biomed. Eng.*, vol. 51, no. 7, pp. 1196–1206, 2004.

[7] R. J. Martis, U. R. Acharya, and L. C. Min, "ECG beat classification using PCA, LDA, ICA and discrete wavelet transform," *Biomed. Signal Process. Control*, vol. 8, no. 5, pp. 437–448, 2013.

[8] T. Li and M. Zhou, "ECG classification using wavelet packet entropy and random forests," *Entropy*, vol. 18, no. 8, Art. no. 285, 2016.

[9] S. Kiranyaz, T. Ince, and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," *IEEE Trans. Biomed. Eng.*, vol. 63, no. 3, pp. 664–675, 2016.

[10] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Adam, A. Gertych, and R. S. Tan, "A deep convolutional neural network model to classify heartbeats," *Comput. Biol. Med.*, vol. 89, pp. 389–396, 2017.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[12] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng, "Cardiologist-level arrhythmia detection and classification in

ambulatory electrocardiograms using a deep neural network," *Nat. Med.*, vol. 25, no. 1, pp. 65–69, 2019.

[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] Ö. Yildirim, "A novel wavelet sequence based on deep bidirectional LSTM network model for ECG signal classification," *Comput. Biol. Med.*, vol. 96, pp. 189–202, 2018.

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[16] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

[17] F. Chollet *et al.*, "Keras," 2015. [Online]. Available: https://keras.io

[18] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[19] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1–5, 2017.

[20] Assistance was received from Claude AI during the project.